

Análisis Técnico - Módulo de Administración de Reservas

Feature: Consulta y Creación Manual de Reservas (HU-01 y HU-02)

Fecha del análisis: 2026-04-08

Historias de Usuario: HU-01 (Consultar Reservas), HU-02 (Crear Reserva Manual)

Organización: Por Servicio (Backend) y por Módulo (Frontend)

Enfoque: Desglose de SOLID y Patrones por Servicio

Introducción

Las Historias de Usuario HU-01 y HU-02 se materializan en una arquitectura de múltiples servicios. Este documento desglosa principios SOLID y patrones de diseño **organizados por servicio**, permitiendo comprender cómo cada servicio contribuye a la funcionalidad del módulo de administración.

PARTE I: BACKEND

1. BOOKINGS-SERVICE

Principal: Gestión de Reservas Administrativas

El bookings-service es responsable central de la consulta paginada y creación manual de reservas. Implementa lógica de validación, transaccionalidad y orquestación de eventos.

1.1 PRINCIPIOS SOLID EN BOOKINGS-SERVICE

****S - Single Responsibility Principle****

Segregación de Responsabilidades por Artefacto:

| Artefacto | Responsabilidad | Archivo | Líneas |

	AdminListReservationsQuery		Encapsular	parámetros	de	consulta	administrativa	
application/usecase/AdminListReservationsQuery.java		1-12						

BookingController.listAdminReservations() Traducir parámetros HTTP a casos de uso
adapters/in/web/BookingController.java 83-114

BookingApplicationService.listAdminReservations() Orquestar lógica de negocio de consulta application/service/BookingApplicationService.java N/A
--

JdbcReservationPersistenceAdapter.listAdminReservations()	Ejecutar query SQL e hidratar objetos
adapters/out/persistence/JdbcReservationPersistenceAdapter.java	N/A

AdminReservationsPageResponse	Estructurar	respuesta	paginada
adapters/in/web/dto/AdminReservationsPageResponse.java	1-12		

BookingHttpMapper.toAdminResponse()	Mapear	Reservation	→	AdminReservationResponse	
adapters/in/web/BookingHttpMapper.java	N/A				

CreateReservationCommand	Encapsular	parámetros	de	creación
application/usecase/CreateReservationCommand.java	1-42			

ReservationStatusCatalog	Normalizar	y	validar	estados	de	reserva
domain/model/ReservationStatusCatalog.java	1-51					

Detalle SRP por Responsabilidad:

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ SINGLE  
RESPONSIBILITY: Consulta Administrativa ■  
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■  
AdminListReservationsQuery ■ ■ ■ ÚNICA RESPONSABILIDAD: Encapsular parámetros filtrados ■ ■ -  
fromExecutionDate, toExecutionDate (rango) ■ ■ - status (estado normalizado) ■ ■ - userId, siteId  
(filtros) ■ ■ - page, size (paginación) ■ ■ - NO contiene lógica, NO accede BD, NO formatea  
respuesta ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■  
■ ■ BookingController.listAdminReservations() ■ ■ ■ ÚNICA RESPONSABILIDAD: Traducir protocolo  
HTTP ■ ■ - Recibe parámetros HTTP (String desde, hasta, etc) ■ ■ - Construye  
AdminListReservationsQuery ■ ■ - Delega a BookingUseCase ■ ■ - Calcula metadatos de paginación ■  
■ ■ - NO contiene lógica de negocio ■ ■ ■  
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■
```

[illegible]

****O - Open/Closed Principle****

Abierto para Extensión, Cerrado para Modificación:

[illegible]

Ejemplo en el Código - ReservationStatusCatalog:

```
// ABIERTO para extensión: nuevos estados pueden agregarse private static final Map<String,
String> ADMIN_TO_SYSTEM_STATUS = Map.of( "pendiente", "pending", "confirmada", "confirmed",
"cancelada", "cancelled", "finalizada", "completed" // AQUÍ: si requiero "en_reprogramacion",
agrego sin modificar lógica ); // Método que respeta OCP: nuevo estado se maneja automáticamente
public static Optional<String> mapAdminToSystem(String adminStatus) { String normalized =
normalizeStatusOrNull(adminStatus); return
Optional.ofNullable(ADMIN_TO_SYSTEM_STATUS.get(normalized)); }
```

****L - Liskov Substitution Principle****

Adaptadores Intercambiables:

CONTRATO: BookingPersistencePort Implementación 1: JdbcReservationPersistenceAdapter ■■ public List<Reservation> listAdminReservations(AdminListReservationsQuery) ■■ Usa JDBC + MariaDB ■■ Retorna List<Reservation> Implementación 2: MockReservationPersistenceAdapter (para testing) ■■ public List<Reservation> listAdminReservations(AdminListReservationsQuery) ■■ Retorna datos en

```
memoria ■■ Retorna List<Reservation> PRINCIPIO LSP: ■■ Ambas implementaciones respetan el
contrato ■■ Cliente (BookingApplicationService) puede usar indistintamente ■■ No hay sorpresas en
el comportamiento contractual GARANTÍA: ■■ Si cambio de JDBC a JPA, el resto del código no se
entera ■■ Si añado una implementación con Oracle, funciona igual ■■ Las transiciones son
transparentes
```

En el Código - ReservationEventPublisherPort:

```
CONTRATO: ReservationEventPublisherPort
Implementación 1: RabbitReservationEventPublisherAdapter
■■ Publica eventos a RabbitMQ cuando RABBITMQ_ENABLED=true ■■ void
publishReservationCreated(ReservationCreatedEvent)
Implementación 2:
NoOpReservationEventPublisherAdapter ■■ No publica nada cuando RABBITMQ_ENABLED=false ■■ void
publishReservationCreated(ReservationCreatedEvent) ■■ return; // sin-op CLIENTE
(BookingApplicationService): ■■ private final ReservationEventPublisherPort eventPublisher; ■■
eventPublisher.publishReservationCreated(event); ■■ ¿Cuál implementación? No le importa, ambas
cumplen el contrato
```

****I - Interface Segregation Principle****

Interfaces Granulares, No Monolíticas:

[illegible]

APLICACIÓN ISP (Bien):

[illegible]

En el Código - Segregación de Puertos:

```
// ARCHIVO: application/port/out/BookingPersistencePort.java // RESPONSABILIDAD: Persistencia de
datos public interface BookingPersistencePort { Reservation create(...); Optional<Reservation>
findById(Long id); List<Reservation> listAdminReservations(AdminListReservationsQuery query); //
Solo métodos de persistencia } // ARCHIVO: application/port/out/ReservationEventPublisherPort.java
// RESPONSABILIDAD: Publicación de eventos public interface ReservationEventPublisherPort { void
publishReservationCreated(ReservationCreatedEvent event); void
publishReservationCancelled(ReservationCancelledEvent event); // Solo métodos de eventos } //
ARCHIVO: application/port/out/TokenPort.java // RESPONSABILIDAD: Validación de tokens public
interface TokenPort { String generate(User user); User validate(String token); // Solo métodos de
tokens } // CLIENTE: @Service public class BookingApplicationService { // Depende de 3 interfaces
segregadas private final BookingPersistencePort persistencePort; private final
ReservationEventPublisherPort eventPublisher; private final TokenPort tokenPort; public
Reservation createReservation(CreateReservationCommand command) { // Usa cada puerto para su
propósito específico persistencePort.create(...); eventPublisher.publishReservationCreated(...);
tokenPort.validate(...); } }
```

****D - Dependency Inversion Principle****

Depender de Abstracciones, No de Concreciones:

DIAGRAMA DE FLUJO - INVERSIÓN DE DEPENDENCIAS:

[illegible]

Inyección de Dependencias en Constructor:

```
// ARCHIVO: application/service/BookingApplicationService.java @Service public class
BookingApplicationService implements BookingUseCase { // Dependencias inyectadas (abstracciones)
private final BookingPersistencePort bookingPersistencePort; private final
ReservationEventPublisherPort eventPublisherPort; private final TokenPort tokenPort; //
Constructor: recibe implementaciones concretas de las abstracciones public
BookingApplicationService( BookingPersistencePort bookingPersistencePort,
ReservationEventPublisherPort eventPublisherPort, TokenPort tokenPort ) {
this.bookingPersistencePort = bookingPersistencePort; this.eventPublisherPort =
eventPublisherPort; this.tokenPort = tokenPort; } @Override public Reservation
createReservation(CreateReservationCommand command) { // La aplicación solo conoce los puertos
(abstracciones) // No importa qué implementaciones específicas vienen inyectadas Reservation
reservation = bookingPersistencePort.create(...);
eventPublisherPort.publishReservationCreated(...); return reservation; } } // EN TIEMPO DE
EJECUCIÓN (Spring Boot): // Spring configura inyecciones basadas en disponibilidad: // Si
RABBITMQ_ENABLED=true: @Configuration public class BookingServiceConfig { @Bean public
ReservationEventPublisherPort eventPublisher() { return new
RabbitReservationEventPublisherAdapter(...); // Concreto } } // Si RABBITMQ_ENABLED=false: @Bean
@ConditionalOnProperty(name = "app.rabbit.enabled", havingValue = "false") public
ReservationEventPublisherPort eventPublisher() { return new
NoOpReservationEventPublisherAdapter(); // Concreto } // RESULTADO: // BookingApplicationService
recibe la implementación apropiada // Pero SIEMPRE ve a través de la abstracción
ReservationEventPublisherPort // La inversión está completa: la aplicación controla qué se inyecta
```

1.2 PATRONES DE DISEÑO EN BOOKINGS-SERVICE

****1. Hexagonal Architecture (Ports & Adapters)****

Estructura de Carpetas - Materialización de Hexagonal:

```

bookings-service/src/main/java/com/reservas/sk/bookings_service/domain/ ← NÚCLEO PURO (sin
dependencias externas) ■■■ model/ ■ ■■■ Reservation.java (Entidad de dominio) ■ ■■■
ReservationEquipment.java (Value Object) ■ ■■■ SpaceAvailability.java (Value Object) ■ ■■■
ReservationStatusCatalog.java (Catálogo de estados) ■■■ service/ ■■■ DateTimeService.java
(Servicio de dominio) application/ ← LÓGICA DE NEGOCIO (orquestación) ■■■ port/ ■ ■■■ in/ ■ ■
■■■ BookingUseCase.java (Interfaz de entrada) ■ ■■■ out/ ■ ■■■ BookingPersistencePort.java ■
■■■ ReservationEventPublisherPort.java ■ ■■■ TokenPort.java ■■■ service/ ■ ■■■
BookingApplicationService.java (Implementación de casos de uso) ■■■ usecase/ ■■■
CreateReservationCommand.java ■■■ AdminListReservationsQuery.java ■■■ ListReservationsQuery.java
■■■ UpdateReservationCommand.java ■■■ HandoverReservationCommand.java ■■■
ReservationCreatedEvent.java ■■■ ... (otros comandos/queries/eventos) adapters/ ← CONVERTIDORES
(hacia frameworks/librerías) ■■■ in/ ■ ■■■ web/ ■ ■■■ BookingController.java (HTTP → Casos de
uso) ■ ■■■ BookingHttpMapper.java (DTO ↔ Dominio) ■ ■■■ HealthController.java ■ ■■■ dto/ ■ ■■■
CreateReservationRequest.java ■ ■■■ AdminReservationsPageResponse.java ■ ■■■
ReservationResponse.java ■ ■■■ AdminReservationResponse.java ■ ■■■ ... (otros DTOs) ■■■ out/ ■
■■■ persistence/ ■ ■ ■■■ JdbcReservationPersistenceAdapter.java (SQL ← Dominio) ■ ■■■ security/
■ ■ ■■■ JwtTokenAdapter.java (JWT ← Dominio) ■ ■■■ messaging/ ■ ■ ■■■
RabbitReservationEventPublisherAdapter.java ■ ■ ■■■ NoOpReservationEventPublisherAdapter.java ■
■■■ websocket/ ■ ■■■ StompReservationRealtimeAdapter.java infrastructure/ ← CONFIGURACIÓN
TÉCNICA ■■■ config/ ■ ■■■ SecurityConfig.java ■ ■■■ WebSocketConfig.java ■ ■■■
WebSocketProperties.java ■ ■■■ JwtProperties.java ■ ■■■ RabbitProperties.java ■■■ security/ ■
■■■ JwtAuthenticationFilter.java ■■■ exception/ ■■■ ApiException.java ■■■
GlobalExceptionHandler.java resources/ ■■■ application.properties ← Configuración externalized

```

Flujo de Solicitud - A través de Capas Hexagonales:

```
HTTP Request █ ▼ ██████████ ADAPTER IN:  
BookingController.java █ █ Validar con @Valid █ Traduce @RequestParam → comando █  
Delega a BookingUseCase █ ██████████ ▼  
██████████ APPLICATION LAYER:  
BookingApplicationService █ Validar reglas de negocio █ Usa ReservationStatusCatalog  
para estados █ Invoca puertos (abstracciones) █ Orquesta flujo █  
██████████ █ ▼  
▼ ██████████ ADAPTER OUT: █ ADAPTER OUT: █ Persistence  
Event Publisher █ (JDBC/SQL) █ (RabbitMQ) █  
██████████ ▼ DOMAIN LAYER (Reservation, entidades puras) Response = Mapper  
(Dominio → DTO) █ ▼ HTTP Response (JSON)
```

Ventajas de Hexagonal en este Servicio:

| Ventaja | Materialización en Código |

|-----|-----|

| **Dominio independiente de Spring** | Reservation sin @Entity, @Column, @JoinTable |

| **Fácil testear negocio** | Mock de puertos en tests, aplicación testeable sin BD |

| **Cambiar BD sin reescribir lógica** | JDBC → JPA: solo cambiar adapter, aplicación igual |

| **Cambiar eventos sin afectar dominio** | RabbitMQ → Kafka: solo cambiar publisher adapter |

| **Evolución sin acoplamiento** | Nuevos filtros en AdminListReservationsQuery sin cambiar adaptadores |

****2. Command Query Responsibility Segregation (CQRS)****

Segregación de Operaciones de Lectura y Escritura:

CQRS en Bookings-Service: QUERIES (Lectura - Sin efectos secundarios) ■■■■ ListReservationsQuery ■■■
■ ■■ Parámetros: userId, spaceId, status ■ ■■ Retorna: List<Reservation> ■ ■■ Handler:
bookingUseCase.listReservations(query) ■ ■■■■ AdminListReservationsQuery ■ ■■ Parámetros:
fromExecutionDate, toExecutionDate, status, userId, siteId, page, size ■ ■■ Retorna:
AdminReservationsPageResponse (paginado) ■ ■■ Handler:
bookingUseCase.listAdminReservations(query) ■ ■■■■ CheckSpaceAvailabilityQuery ■ ■■ Parámetros:
spaceId, startAt, endAt ■ ■■ Retorna: SpaceAvailability (slots ocupados) ■ ■■ Handler:
bookingUseCase.checkAvailability(query) ■ ■■■■ AdminListReservationsQuery (conteo) ■ ■■ Parámetros:
mismos que AdminListReservationsQuery ■ ■■ Retorna: Long (total de items) ■ ■■ Handler:
bookingUseCase.countAdminReservations(query) COMMANDS (Escritura - Con efectos secundarios) ■■■■
CreateReservationCommand ■ ■■ Parámetros: userId, spaceId, startAt, endAt, equipmentIds,
attendeesCount ■ ■■ Retorna: Reservation ■ ■■ Handler: bookingUseCase.createReservation(command)
■ ■■ Efectos: Inserta reserva, publica ReservationCreatedEvent ■ ■■■■ UpdateReservationCommand ■
■ ■■ Parámetros: id, userId, title, startAt, endAt, attendeesCount, notes ■ ■■ Retorna: Reservation
■ ■■ Handler: bookingUseCase.updateReservation(command) ■ ■■■■ HandoverReservationCommand ■ ■■
Parámetros: reservationId, userId, novelty ■ ■■ Retorna: Reservation ■ ■■ Handler:
bookingUseCase.deliverReservation(command) o returnReservation(command) ■ ■■ Efectos: Cambia
estado, publica evento de entrega/devolución ■ ■■■■ CancelCommand (implícito) ■ ■■ Parámetros:
reservationId, reason ■ ■■ Retorna: Reservation ■ ■■ Handler:
bookingUseCase.cancelReservation(reservationId, reason) ■ ■■ Efectos: Cambia estado a CANCELLED,
publica ReservationCancelledEvent BENEFICIOS CQRS: ■ ■■ Lectura: Optimizada en SQL (índices en
start_datetime) ■ ■■ Escritura: Validaciones y eventos complejos ■ ■■ Escalabilidad: Pueden crecer
independientemente ■ ■■ Testing: Queries testean sin afectar datos; Commands testean lógica ■ ■■
Auditabilidad: Cada Command es un evento de negocio registrable

Implementación en Controller:

```
// ARCHIVO: BookingController.java // ===== QUERIES ===== // Query
1: Listar reservas del usuario regular @GetMapping("/reservations") public
ApiResponse<List<ReservationResponse>> listReservations( @RequestParam(required = false) Long
userId, @RequestParam(required = false) Long spaceId, @RequestParam(required = false) String
status ) { var reservations = bookingUseCase.listReservations( new ListReservationsQuery(userId,
spaceId, status) ); return ApiResponse.success(reservations.stream() .map(mapper::toResponse)
.toList()); } // Query 2: Consultar disponibilidad @GetMapping("/spaces/{spaceId}/availability")
public ApiResponse<SpaceAvailabilityResponse> checkAvailability( @PathVariable Long spaceId,
@RequestParam String startAt, @RequestParam String endAt ) { var availability =
bookingUseCase.checkAvailability( new CheckSpaceAvailabilityQuery(spaceId, startAt, endAt) );
return ApiResponse.success(mapper.toAvailabilityResponse(availability)); } // Query 3: Listar
reservas administrativas (paginadas) @GetMapping("/admin/reservations") public
ApiResponse<AdminReservationsPageResponse> listAdminReservations( @RequestParam(required = false)
String desde, @RequestParam(required = false) String hasta, @RequestParam(required = false) String
estado, @RequestParam(required = false) Long usuario, @RequestParam(required = false) Long sede,
@RequestParam(required = false, defaultValue = "0") Integer page, @RequestParam(required = false,
```

```

defaultValue = "20") Integer size ) { var query = new AdminListReservationsQuery(desde, hasta,
estado, usuario, sede, page, size); var reservations =
bookingUseCase.listAdminReservations(query); long totalItems =
bookingUseCase.countAdminReservations(query); // Calcula metadatos de paginación int safeSize =
size == null || size <= 0 ? 20 : size; int safePage = page == null || page < 0 ? 0 : page; int
totalPages = totalItems == 0 ? 0 : (int) Math.ceil((double) totalItems / safeSize); return
ApiResponse.success(new AdminReservationsPageResponse(
reservations.stream().map(mapper::toAdminResponse).toList(), safePage, safeSize, totalItems,
totalPages )); } // ===== COMMANDS ===== // Command 1: Crear reserva
(usuario o admin) @PostMapping("/reservations") public
ResponseEntity<ApiResponse<ReservationResponse>> createReservation( @Valid @RequestBody
CreateReservationRequest request, @AuthenticationPrincipal AuthenticatedUser user ) { Long
effectiveUserId = resolveEffectiveUserId(user, request.targetUserId()); var reservation =
bookingUseCase.createReservation( new CreateReservationCommand( effectiveUserId,
request.spaceId(), request.startAt(), request.endAt(), request.title(), request.attendeesCount(),
request.notes(), request.equipmentIds() ) ); return ResponseEntity.status(HttpStatus.CREATED)
.body(ApiResponse.success(mapper.toResponse(reservation))); } // Command 2: Actualizar reserva
@PutMapping("/reservations/{id}") public ResponseEntity<ApiResponse<ReservationResponse>>
updateReservation( @PathVariable Long id, @Valid @RequestBody UpdateReservationRequest request,
@AuthenticationPrincipal AuthenticatedUser user ) { var reservation =
bookingUseCase.updateReservation( new UpdateReservationCommand( id, user.userId(),
request.title(), request.startAt(), request.endAt(), request.attendeesCount(), request.notes() )
); return ResponseEntity.ok(ApiResponse.success(mapper.toResponse(reservation))); } // Command 3:
Cancelar reserva @PatchMapping("/reservations/{id}/cancel") public
ApiResponse<ReservationResponse> cancel( @PathVariable Long id, @RequestBody(required = false)
CancelReservationRequest request ) { String reason = request == null ? null : request.reason();
return ApiResponse.success(mapper.toResponse( bookingUseCase.cancelReservation(id, reason) )); }
// Command 4: Entregar reserva (cambio de estado) @PatchMapping("/reservations/{id}/deliver")
public ApiResponse<ReservationResponse> deliver( @PathVariable Long id, @RequestBody(required =
false) HandoverReservationRequest request, @AuthenticationPrincipal AuthenticatedUser user ) {
String novelty = request == null ? null : request.novelty(); return
ApiResponse.success(mapper.toResponse( bookingUseCase.deliverReservation( new
HandoverReservationCommand(id, user.userId(), novelty) ) )); }

```

****3. Adapter Pattern (Input/Output)****

Adaptador de Entrada - HTTP:

```
// ARCHIVO: adapters/in/web/BookingController.java // RESPONSABILIDAD: Traducir HTTP → Dominio
@RestController @RequestMapping("/bookings") public class BookingController { private final
BookingUseCase bookingUseCase; private final BookingHttpMapper mapper; //
// Adapter IN: HTTP → Dominio //
// @GetMapping("/admin/reservations") public
ApiResponse<AdminReservationsPageResponse> listAdminReservations( @RequestParam(required = false)
String desde, // String HTTP @RequestParam(required = false) String hasta, @RequestParam(required =
false) String estado, @RequestParam(required = false) Long usuario, @RequestParam(required =
false) Long sede, @RequestParam(required = false, defaultValue = "0") Integer page,
@RequestParam(required = false, defaultValue = "20") Integer size ) { // TRADUCCIÓN: parámetros
HTTP → Objeto de caso de uso var normalizedQuery = new AdminListReservationsQuery( desde, // Sin
cambios, backend validará hasta, estado, // Se normalizará en aplicación usuario, sede, page, size
); // INVOCACIÓN: delega a caso de uso var reservations =
bookingUseCase.listAdminReservations(normalizedQuery); long totalItems =
bookingUseCase.countAdminReservations(normalizedQuery); // CÁLCULO DE METADATOS: paginación int
safeSize = size == null || size <= 0 ? 20 : size; int safePage = page == null || page < 0 ? 0 :
page; int totalPages = totalItems == 0 ? 0 : (int) Math.ceil((double) totalItems / safeSize); //
MAPEO: Dominio → DTO HTTP return ApiResponse.success(new AdminReservationsPageResponse(
reservations.stream().map(mapper::toAdminResponse) // Reservation → AdminReservationResponse
.toList(), safePage, safeSize, totalItems, totalPages )); } }
```

Adaptador de Salida - Persistencia:

```
// ARCHIVO: adapters/out/persistence/JdbcReservationPersistenceAdapter.java // RESPONSABILIDAD:
Traducir Dominio ↔ SQL/BD @Component public class JdbcReservationPersistenceAdapter implements
BookingPersistencePort { private final JdbcTemplate jdbcTemplate; //
// Adapter OUT: SQL/BD → Dominio //
@Override public List<Reservation>
listAdminReservations(AdminListReservationsQuery query) { // CONSTRUCCIÓN: Query SQL dinámica
basada en criterios StringBuilder sql = new StringBuilder( "" SELECT r.*, u.name, u.email, c.name
as city_name, s.name as space_name FROM reservations r JOIN users u ON r.user_id = u.id JOIN cities
c ON r.city_id = c.id JOIN spaces s ON r.space_id = s.id WHERE 1=1 "" ); List<Object> params = new
ArrayList<>(); // FILTRADO DINÁMICO if (query.fromExecutionDate() != null) { sql.append(" AND
r.start_datetime >= ?"); params.add(Timestamp.valueOf(query.fromExecutionDate() + " 00:00:00")); }
if (query.toExecutionDate() != null) { sql.append(" AND r.start_datetime <= ?");
params.add(Timestamp.valueOf(query.toExecutionDate() + " 23:59:59")); } if (query.status() !=
null) { sql.append(" AND r.status = ?"); params.add(query.status()); } if (query.userId() != null)
{ sql.append(" AND r.user_id = ?"); params.add(query.userId()); } if (query.siteId() != null) {
sql.append(" AND c.id = ?"); params.add(query.siteId()); } // ORDENAMIENTO sql.append(" ORDER BY
r.start_datetime DESC"); // PAGINACIÓN sql.append(" LIMIT ? OFFSET ?"); params.add(query.size());
params.add(query.page() * query.size()); // EJECUCIÓN: Query en BD return jdbcTemplate.query(
sql.toString(), (rs, rowNum) -> toReservation(rs), // Mapeo ResultSet → Objeto params.toArray() );
} // MAPEO: ResultSet → Dominio private Reservation toReservation(ResultSet rs) throws
```


****4. Strategy Pattern****

Estrategia de Resolución de Usuario (Admin Override):

```
// ARCHIVO: adapters/in/web/BookingController.java private Long
resolveEffectiveUserId(AuthenticatedUser user, Long targetUserId) { // ESTRATEGIA 1: Sin override
(usuario regular) if (targetUserId == null) { return user.userId(); // Crea reserva para sí mismo }
// ESTRATEGIA 2: Override por administrador if (user.hasRole("ADMIN")) { return targetUserId; //
Admin crea reserva para otro usuario } // ESTRATEGIA 3: Intento de override no autorizado if
(!user.userId().equals(targetUserId)) { throw new ApiException( HttpStatus.FORBIDDEN, "No tiene
permisos para crear reservas para otros usuarios", "FORBIDDEN_TARGET_USER" ); } // Fallback return
user.userId(); } // USO EN CONTROLLER: @PostMapping("/reservations") public
ResponseEntity<ApiResponse<ReservationResponse>> createReservation( @Valid @RequestBody
CreateReservationRequest request, @AuthenticationPrincipal AuthenticatedUser user ) { // Aplica
estrategia de resolución Long effectiveUserId = resolveEffectiveUserId(user,
request.targetUserId()); // Usa userId resuelto var reservation =
bookingUseCase.createReservation( new CreateReservationCommand( effectiveUserId, // ← Usuario
calculado por estrategia request.spaceId(), ... ) ); return
ResponseEntity.status(HttpStatus.CREATED)
.body(ApiResponse.success(mapper.toResponse(reservation))); }
```

Estrategia de Publicación de Eventos:

CONTEXTO: ¿Cómo publicar eventos? ¿RabbitMQ o sin-op? ESTRATEGIA 1: RabbitMQ (Concreto) @Component @ConditionalOnProperty(prefix = "app.rabbit", name = "enabled", havingValue = "true") public class RabbitReservationEventPublisherAdapter implements ReservationEventPublisherPort { // Publica eventos a RabbitMQ } ESTRATEGIA 2: No-op (Concreto) @Component @ConditionalOnProperty(prefix = "app.rabbit", name = "enabled", havingValue = "false", matchIfMissing = true) public class NoOpReservationEventPublisherAdapter implements ReservationEventPublisherPort { // No publica nada }

INTERFAZ (Abstracción): public interface ReservationEventPublisherPort { void publishReservationCreated(ReservationCreatedEvent event); }

CLIENTE (BookingApplicationService): @Service public class BookingApplicationService { private final ReservationEventPublisherPort eventPublisherPort; // ← Recibe cualquier estrategia public Reservation createReservation(CreateReservationCommand command) { ... Reservation reservation = persistencePort.create(...); // Invoca sin conocer qué estrategia está activa eventPublisherPort.publishReservationCreated(new ReservationCreatedEvent(...)); return reservation; } }

BENEFICIO: ■■ En desarrollo: RabbitMQ deshabilitado (NoOp, más rápido) ■■ En producción: RabbitMQ habilitado (Rabbit, eventos reales) ■■ Código aplicación: idéntico en ambos casos ■■ Cambio de estrategia: solo configuración, sin recompilación

****5. Validation Chain Pattern****

Cadena de Validaciones Encadenadas:

[illegible]

Diagrama de Flujo - Validation Chain:

```

##### CreateReservationCommand recibido #####
##### ▼ ##### Validación ##### 1:
Espacio ##### ¿Existe? ##### NO → throw ApiException(NOT_FOUND) → HTTP 404 ##### Sí
↓ ##### Validación 2: ##### Rango de fechas #####
##### ¿startAt < endAt? ##### NO → throw ApiException(BAD_REQUEST) → HTTP 400 ##### Sí ↓
##### Validación 3: ##### Sin solapamiento #####
##### ¿Hay conflicto? ##### Sí → throw ApiException(CONFLICT) → HTTP 409
##### NO ↓ ##### Validación 4: ##### Equipos válidos #####

```

```
##### ¿Equipos OK? ■ NO → throw ApiException(BAD_REQUEST) → HTTP 400
■ ■ Sí ↓ ##### ✓ TODAS LAS ■ VALIDACIONES ■ PASARON ■
##### ▼ ##### 1. INSERT en BD ■ 2.
Publica evento ■ 3. Retorna objeto ■ #####
```

****6. DTO (Data Transfer Object) Pattern****

DTOs Segregados por Contexto:

```
// ████████████████████████████████████████████████████████████████████████████████ // USUARIO REGULAR  
→ Ver solo reservas propias //  
██████████████████████████████████████████████████████████████████████████████ public record  
ReservationResponse( Long id, Long userId, Long spaceId, String spaceName, Instant startDatetime,  
Instant endDatetime, String status, String title, Integer attendeesCount, String notes,  
List<ReservationEquipmentResponse> equipments, Instant createdAt ) //  
██████████████████████████████████████████████████████████████████████████████ // ADMINISTRADOR → Ver  
todas las reservas + metadata //  
██████████████████████████████████████████████████████████████████████████████ public record  
AdminReservationResponse( Long id, Long userId, String userName, // ← Extra: nombre del usuario  
String userEmail, // ← Extra: email del usuario Long siteId, String siteName, Long spaceId, String  
spaceName, Instant executionDate, // ← Fecha de ejecución (no creación) String startTime, // ←  
Hora extractada String endTime, // ← Hora extractada String status, Integer attendeesCount, String  
notes, List<ReservationEquipmentResponse> equipment, Instant createdAt ) public record  
AdminReservationsPageResponse( List<AdminReservationResponse> items, // ← Items de la página  
Integer page, // ← Página actual Integer size, // ← Tamaño de página Long totalItems, // ← Total  
de registros Integer totalPages // ← Total de páginas ) // MAPEO: Dominio → DTOs public class  
BookingHttpMapper { // Mapeo para usuario regular public ReservationResponse  
toResponse(Reservation reservation) { return new ReservationResponse( reservation.getId(),  
reservation.getUserId(), reservation.getSpaceId(), reservation.getSpaceName(),  
reservation.getStartDatetime(), reservation.getEndDatetime(), reservation.getStatus(),  
reservation.getTitle(), reservation.getAttendeesCount(), reservation.getNotes(),  
mapEquipments(reservation.getEquipments()), reservation.getCreatedAt() ); } // Mapeo para  
administrador public AdminReservationResponse toAdminResponse(Reservation reservation) { return  
new AdminReservationResponse( reservation.getId(), reservation.getUserId(),  
reservation.getUserName(), // ← Información extra del usuario reservation.getUserEmail(),  
reservation.getSiteId(), reservation.getSiteName(), reservation.getSpaceId(),  
reservation.getSpaceName(), reservation.getStartDatetime(), // ← executionDate  
extractTime(reservation.getStartDatetime()), extractTime(reservation.getEndDatetime()),  
reservation.getStatus(), reservation.getAttendeesCount(), reservation.getNotes(),  
mapEquipments(reservation.getEquipments()), reservation.getCreatedAt() ); } private String  
extractTime(Instant instant) { return instant.toString().substring(11,19); // "HH:mm:ss" } }
```

****7. Null Object Pattern****

Implementación NoOp para Publicación de Eventos:

[illegible]

2. AUTH-SERVICE

Soporte: Validación de Tokens JWT

El auth-service proporciona mecanismos de autenticación y generación de tokens. En el contexto del módulo admin, participa en:

- Validación de tokens JWT en headers Authorization
- Extracción de roles del usuario (admin vs regular)

2.1 PRINCIPIOS SOLID EN AUTH-SERVICE

****D - Dependency Inversion Principle****

CONTEXTO: ¿Cómo validar tokens JWT? ABSTRACCIÓN: `TokenPort` `+ generate(user): String` `+ validate(token): User` IMPLEMENTACIONES CONCRETAS: `JwtTokenAdapter` `Usa JJWT library` `Firma con RS256 (RSA)` `OAuthTokenAdapter (futura)` `Usa OAuth provider` `Delega validación a tercero CLIENTE (JwtAuthenticationFilter, BookingApplicationService)` `Depende de TokenPort (abstracción)` `No importa qué implementación está inyectada` `Cambiar JWT a OAuth: solo cambiar adapter, código cliente igual`

****S - Single Responsibility Principle****

`JwtTokenAdapter` **ÚNICA RESPONSABILIDAD: Generar/validar JWT** `- Firma tokens con secret RSA` `- Extrae claims del token` `- Valida expiración` `- NO valida roles (responsabilidad de SecurityConfig)` `- NO accede a BD` `- NO publica eventos`

2.2 PATRONES EN AUTH-SERVICE

****1. Adapter Pattern (JWT)****

```
// ARCHIVO: adapters/out/security/JwtTokenAdapter.java @Component public class JwtTokenAdapter
implements TokenPort { private final JwtProperties jwtProperties; @Override public String
generate(User user) { // GENERACIÓN: Crea token JWT con claims return Jwts.builder()
.setSubject(user.getId().toString()) // sub: user_id .claim("email", user.getEmail()) // email:
user@example.com .claim("roles", user.getRoles()) // roles: [USER, ADMIN] .setIssuedAt(new Date())
.setExpiration(new Date(System.currentTimeMillis() + jwtProperties.getExpirationMs()))
.signWith(SignatureAlgorithm.HS256, jwtProperties.getSecret()) .compact(); } @Override public User
validate(String token) { // VALIDACIÓN: Parsea y valida token Claims claims = Jwts.parser()
.setSigningKey(jwtProperties.getSecret()) .parseClaimsJws(token) .getBody(); // EXTRACCIÓN: Claims
→ Objeto User return new User( Long.parseLong(claims.getSubject()), claims.get("email",
String.class), claims.get("roles", List.class) ); } } // ARCHIVO:
infrastructure/security/JwtAuthenticationFilter.java @Component public class
JwtAuthenticationFilter extends OncePerRequestFilter { private final TokenPort tokenPort;
@Override protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain filterChain) throws ServletException, IOException { // EXTRACCIÓN: Bearer
token del header String authHeader = request.getHeader("Authorization"); if (authHeader != null &&
authHeader.startsWith("Bearer ")) { String token = authHeader.substring(7); try { // VALIDACIÓN:
TokenPort valida User user = tokenPort.validate(token); // CONTEXTO: Establece usuario autenticado
en SecurityContext SecurityContext context = SecurityContextHolder.createEmptyContext();
context.setAuthentication( new JwtAuthenticationToken(user) );
SecurityContextHolder.setContext(context); } catch (JwtException e) { // Token inválido o expirado
SecurityContextHolder.clearContext(); } } filterChain.doFilter(request, response); } }
```

3. LOCATIONS-SERVICE

Soporte: Catálogo de Ciudades y Espacios

El locations-service proporciona:

- Listado de ciudades/sedes disponibles
- Listado de espacios por ciudad
- Validación de existencia de ciudades/espacios

3.1 PRINCIPIOS SOLID EN LOCATIONS-SERVICE

****O - Open/Closed Principle****

CONTEXTO: ¿Qué datos expone locations-service? EN FUTURO, podría necesitarse: - Listado de plantas por sede - Listado de pisos - Datos de capacidad DISEÑO OCP: ■■ Cada agregado (City, Space) puede evolucionarse ■■ API admin puede agregar campos sin romper clientes ■■ Nuevos filtros en AdminReservationsPage sin cambiar backend

3.2 PATRONES EN LOCATIONS-SERVICE

****1. Repository Pattern****

```
// ARCHIVO: application/port/out/LocationsPersistencePort.java public interface
LocationsPersistencePort { Optional<City> findCityById(Long id); List<City> listCities();
Optional<Space> findSpaceById(Long id); List<Space> findSpacesByCity(Long cityId); } // ARCHIVO:
adapters/out/persistence/JdbcLocationsPersistenceAdapter.java @Component public class
JdbcLocationsPersistenceAdapter implements LocationsPersistencePort { @Override public List<Space>
findSpacesByCity(Long cityId) { String sql = "" SELECT id, city_id, name, capacity, description
FROM spaces WHERE city_id = ? AND active = true ORDER BY name ASC ""; return jdbcTemplate.query(
sql, (rs, rowNum) -> new Space( rs.getLong("id"), rs.getLong("city_id"), rs.getString("name"),
rs.getInt("capacity"), rs.getString("description" ) , cityId ); } }
```

4. INVENTORY-SERVICE

Soporte: Catálogo de Equipos

El inventory-service proporciona:

- Listado de equipos disponibles por ciudad
- Validación de equipos en creación de reservas
- Estados de equipos (available, maintenance, retired)

4.1 PRINCIPIOS SOLID EN INVENTORY-SERVICE

****S - Single Responsibility Principle****

RESPONSABILIDADES SEGREGADAS: ■ InventoryApplicationService ■ ■■ Validar existencia de equipo ■ ■■ Validar disponibilidad de equipo ■ ■■ Validar pertenencia a ciudad ■ ■■ NO valida disponibilidad horaria (responsabilidad de BookingsService) ■ ■■ NO crea reservas (responsabilidad de BookingsService)

4.2 PATRONES EN INVENTORY-SERVICE

****1. Repository Pattern****

```
// ARCHIVO: application/port/out/InventoryPersistencePort.java public interface
InventoryPersistencePort { Optional<Equipment> findEquipmentById(Long id); List<Equipment>
findEquipmentByCity(Long cityId); boolean cityExists(Long cityId); int
updateEquipmentAvailability(Long equipmentId, String newStatus); } // ARCHIVO:
adapters/out/persistence/JdbcInventoryPersistenceAdapter.java @Component public class
JdbcInventoryPersistenceAdapter implements InventoryPersistencePort { @Override public
List<Equipment> findEquipmentByCity(Long cityId) { String sql = "" SELECT e.id, e.city_id,
e.name, e.status, e.available FROM equipments e WHERE e.city_id = ? AND e.status = 'available'
ORDER BY e.name ASC ""; return jdbcTemplate.query( sql, (rs, rowNum) -> new Equipment(
rs.getLong("id"), rs.getLong("city_id"), rs.getString("name"), rs.getString("status"),
rs.getBoolean("available") ), cityId ); } }
```

5. NOTIFICATIONS-SERVICE

Soporte: Notificaciones en Tiempo Real

El `notifications-service` consume eventos publicados por `Bookings-Service` y los retransmite via `WebSocket STOMP`.

5.1 PATRONES EN NOTIFICATIONS-SERVICE

****1. Observer/Pub-Sub Pattern****

FLUJO: 1. BookingsService crea reserva ■■ Publica: ReservationCreatedEvent ■■ → RabbitMQ exchange (reservas.events) 2. NotificationsService escucha ■■ Consume: ReservationCreatedEvent ■■ Desde cola (notifications.events.q) 3. NotificationsService retransmite ■■ Envía a WebSocket STOMP ■■ Topic: /topic/events.bookings.reservation.created ■■ Payload: ReservationCreatedEvent (JSON) 4. Frontend (Admin) se suscribe ■■
stompClient.subscribe('/topic/events.bookings.reservation.created', callback) ■■ callback recibe evento ■■ Actualiza tabla de reservas en tiempo real

PARTE II: FRONTEND

1. ADMIN MODULE - REACT + TYPESCRIPT

La lógica de administración en frontend se articula alrededor de:

- **AdminReservationsPage**: Orquestador de estado y flujos
- **Componentes presentacionales**: ReservationStatusFilter, AdminCreateReservationPanel
- **Hooks personalizados**: useAdminReservationFilters, useAdminCreateReservationForm
- **Servicios HTTP**: adminReservationsService
- **Mappers**: ReservationMapper, UserMapper

1.1 PRINCIPIOS SOLID EN FRONTEND

****S - Single Responsibility Principle****

COMPONENTES SEGREGADOS POR RESPONSABILIDAD: AdminReservationsPage ■■ RESPONSABILIDAD: Orquestar filtros y listados ■■ Gestiona: estado de filtros, tabla, paginación ■■ Delega a: servicios HTTP, componentes presentacionales ■■ NO renderiza: formularios, filtros directamente
ReservationStatusFilter ■■ RESPONSABILIDAD: Renderizar controles de filtro ■■ Props: filters, setFilter, clearFilters, applyFilters ■■ NO maneja: lógica de búsqueda AdminCreateReservationPanel ■■ RESPONSABILIDAD: Renderizar formulario de creación ■■ Props: form, errors, onFieldChange, onSubmit ■■ NO valida: lógica de validación en hooks useAdminReservationFilters ■■
RESPONSABILIDAD: Gestionar estado de filtros ■■ Retorna: filters, setFilter, clearFilters ■■ NO renderiza: componentes useAdminCreateReservationForm ■■ RESPONSABILIDAD: Gestionar estado de formulario ■■ Retorna: form, errors, setFieldValue, validate ■■ NO renderiza: componentes adminReservationsService ■■ RESPONSABILIDAD: Orquestar llamadas HTTP ■■ Funciones: getAdminReservations, createAdminReservation, searchAdminUsers ■■ Mapea: respuestas HTTP a entidades de dominio

****D - Dependency Inversion Principle****

INYECCIÓN DE DEPENDENCIAS EN FRONTEND: AdminReservationsPage (Contenedor) ■■ Recibe servicios inyectados: ■ ■■ getAdminReservations (función) ■ ■■ createAdminReservation (función) ■ ■■ searchAdminUsers (función) ■ ■■ checkAdminReservationAvailability (función) ■ ■■ Los servicios son abstracciones: ■■ Podrían ser reemplazados por mocks para testing ■■ Podrían cambiar de HTTP a GraphQL ■■ Componente no se entera del cambio BENEFICIO: ■■ Testing: inyectar servicios mock ■■ Evolución: cambiar implementación sin reescribir componente

1.2 PATRONES DE DISEÑO EN FRONTEND

****1. Container/Presentational Component Pattern****

Container Component (Smart):

```
// AdminReservationsPage.jsx // RESPONSABILIDAD: Lógica, estado, efectos secundarios export
function AdminReservationsPage() { // Estado const [reservations, setReservations] = useState([]);
const [totalPages, setTotalPages] = useState(0); const [isCreateModalOpen, setIsCreateModalOpen] =
useState(false); const [successMessage, setSuccessMessage] = useState(null); // Hooks
personalizados const { filters, setFilter, clearFilters } = useAdminReservationFilters(); const {
form, errors, setFieldValue, validate, reset } = useAdminCreateReservationForm(); // Efectos
useEffect(() => { loadAdminReservations(); }, []); // Lógica const loadAdminReservations = async
(page = 0) => { const result = await getAdminReservations({ dateFrom: filters.dateRangeFrom,
dateTo: filters.dateRangeTo, status: filters.status, userId: filters.userId, siteId:
filters.siteId, page, size: 20 }); setReservations(result.items);
setTotalPages(result.totalPages); }; // Renderiza componentes presentacionales (Dumb) return ( <
<ReservationStatusFilter filters={filters} setFilter={setFilter} clearFilters={clearFilters}
applyFilters={() => loadAdminReservations(0)} /> <ReservationTable reservations={reservations}
totalPages={totalPages} onPageChange={(page) => loadAdminReservations(page)}
onRowClick={(reservation) => {...}} /> <AdminCreateReservationPanel isOpen={isCreateModalOpen}
form={form} errors={errors} onFieldChange={setFieldValue} onSubmit={handleCreateReservation}
onClose={() => setIsCreateModalOpen(false)} /> </> ); }
```

Presentational Component (Dumb):

```
// AdminCreateReservationPanel.jsx // RESPONSABILIDAD: Renderización, sin estado de negocio export
const AdminCreateReservationPanel = ({ isOpen, form, errors, onFieldChange, onSubmit, onClose, //
... otras props }) => { // Sin useEffect, sin servicios HTTP // Solo props y renderización if
(!isOpen) return null; return ( <div className="modal-overlay"> <form onSubmit={onSubmit}> <input
value={form.date} onChange={(e) => onFieldChange('date', e.target.value)} /> {errors.date && <span
className="error">{errors.date}</span>} <button type="submit" disabled={Object.keys(errors).length
> 0}> Crear Reserva </button> </form> </div> ); };
```

****2. Custom Hooks Pattern****

Hook para Gestión de Filtros:

```
// core/adapters/hooks/useAdminReservationFilters.ts export function useAdminReservationFilters()
{ // Estado const [filters, setFilters] = useState({ dateRangeFrom: null, dateRangeTo: null,
status: null, userId: null, siteId: null, page: 0 }); // Setters const setFilter =
useCallback((key: string, value: any) => { setFilters(prev => ({ ...prev, [key]: value, page: key
=== 'page' ? value : 0 // Reset page si cambia filtro })); }, []); const clearFilters =
useCallback(() => { setFilters({ dateRangeFrom: null, dateRangeTo: null, status: null, userId:
null, siteId: null, page: 0 }); }, []); return { filters, setFilter, clearFilters }; }
```

Hook para Gestión de Formulario:

```
// core/adapters/hooks/useAdminCreateReservationForm.ts export function
useAdminCreateReservationForm() { // Estado del formulario const [form, setForm] = useState({
userId: null, site: '', space: '', date: '', startTime: '', endTime: '', attendeesCount: 1,
equipment: [] }); // Estado de errores const [errors, setErrors] = useState({}); const
[hasConflict, setHasConflict] = useState(false); // Actualizar campo const setFieldValue =
useCallback((field: string, value: any) => { setForm(prev => ({ ...prev, [field]: value })); //
Limpiar error del campo setErrors(prev => { const newErrors = { ...prev }; delete newErrors[field];
return newErrors; }); }, []); // Validar const validate = useCallback(() => { const newErrors:
Record<string, string> = {}; if (!form.userId) newErrors.userId = 'Usuario requerido'; if
(!form.site) newErrors.site = 'Sede requerida'; if (!form.space) newErrors.space = 'Espacio
requerido'; if (!form.date) newErrors.date = 'Fecha requerida'; if (!form.startTime)
newErrors.startTime = 'Hora inicio requerida'; if (!form.endTime) newErrors.endTime = 'Hora fin
requerida'; if (form.startTime >= form.endTime) { newErrors.endTime = 'Debe ser posterior a
inicio'; } setErrors(newErrors); return Object.keys(newErrors).length === 0; }, [form]); // Reset
const reset = useCallback(() => { setForm({ userId: null, site: '', space: '', date: '', startTime:
'', endTime: '', attendeesCount: 1, equipment: [] }); setErrors({}); setHasConflict(false); },
[]); return { form, setFieldValue, errors, setErrors, hasConflict, setHasConflict, validate, reset
}; }
```

****3. Service Layer Pattern****

Servicio HTTP Centralizado:

```
// features/reservations/services/adminReservationsService.ts export async function
getAdminReservations(query: AdminListReservationsQuery) { const response = await
httpClient.get('/bookings/admin/reservations', { params: { desde: query.dateFrom, hasta:
query.dateTo, estado: query.status, usuario: query.userId, sede: query.siteId, page: query.page,
size: 20 } }); return { items: ReservationMapper.toDomainList(response.data.data.items), page:
response.data.data.page, size: response.data.data.size, totalItems: response.data.data.totalItems,
totalPages: response.data.data.totalPages }; } export async function
createAdminReservation(payload: CreateReservationPayload) { const response = await
httpClient.post('/bookings/reservations', { targetUserId: payload.userId, spaceId:
payload.spaceId, startAt: `${payload.date}T${payload.startTime}`, endAt:
`${payload.date}T${payload.endTime}`, attendeesCount: payload.attendeesCount, equipmentIds:
payload.equipmentIds }); return ReservationMapper.toDomain(response.data.data); } export async
function searchAdminUsers(query: string) { const response = await httpClient.get('/auth/users', {
params: { query } }); return UserMapper.toDomainList(response.data.data); }
```

****4. Mapper Pattern****

```
// infrastructure/mappers/ReservationMapper.ts export class ReservationMapper { // API Response →  
Entidad de Dominio static toDomain(raw: any): Reservation { return new Reservation({ id:  
raw.id?.toString(), userId: raw.userId?.toString(), locationId: raw.spaceId?.toString(),  
locationName: raw.spaceName, startAt: raw.startDatetime, endAt: raw.endDatetime, status:  
raw.status, createdAt: raw.createdAt, attendeesCount: raw.attendeesCount, notes: raw.notes,  
equipment: raw equipments?.map(e => e.name) || [] }); } static toDomainList(raw: any[]):  
Reservation[] { return raw.map(item => this.toDomain(item)); } }
```

1.3 INTEGRACIÓN FRONTEND-BACKEND: FLUJOS COMPLETOS

****Flujo 1: Consulta Paginada (HU-01)****

```
FRONTEND: AdminReservationsPage ■■ Inicial: useEffect → loadAdminReservations(filters={}, page=0)
■■ User input: setFilter('status', 'confirmada') ■■ User action: applyFilters() ■■■ Invoca:
getAdminReservations({status: 'confirmada', page: 0, size: 20}) ■■■ Respuesta: ■■
setReservations(items) ■■ setTotalPages(totalPages) ■■ Renderiza: tabla con 20 reservas, botones
de página HTTP REQUEST: GET /bookings/admin/reservations?estado=confirmada&page;=0&size;=20
Authorization: Bearer <JWT_TOKEN> BACKEND: BookingController.listAdminReservations() ■■ Crea:
AdminListReservationsQuery(estado='confirmada', page=0, size=20) ■■ Invoca:
bookingUseCase.listAdminReservations(query) ■■ Invoca:
bookingUseCase.countAdminReservations(query) ■■ Calcula: totalPages = ceil(totalItems / 20) ■■
Retorna: ApiResponse<AdminReservationsPageResponse> DB QUERY: SELECT r.*, u.name, c.name, s.name
FROM reservations r JOIN users u ON r.user_id = u.id JOIN cities c ON r.city_id = c.id JOIN spaces
s ON r.space_id = s.id WHERE r.status = 'confirmed' ORDER BY r.start_datetime DESC LIMIT 20 OFFSET
0; HTTP RESPONSE: { "ok": true, "data": { "items": [ { "id": 1, "userName": "Juan Pérez",
"siteName": "Sede Medellín", "spaceName": "Sala A1", "executionDate": "2026-04-15", "startTime":
"14:00", "endTime": "15:30", "status": "confirmada", ... }, ... ], "page": 0, "size": 20,
"totalItems": 150, "totalPages": 8 } } FRONTEND (Renderización): ■■ ReservationTable muestra 20
reservas ■■ Pagination muestra: página 1 de 8 ■■ User puede navegar: siguiente, anterior ■■ Al
cambiar página: loadAdminReservations(filters, page=1)
```

****Flujo 2: Creación Manual (HU-02)****

```
FRONTEND: AdminReservationsPage ■■ User: Click "Nueva Reserva" ■■ setIsCreateModalOpen(true) ■■
AdminCreateReservationPanel abre ■■ User ingresa datos: ■■■ Busca usuario:
onUserSearch('Juan') → searchAdminUsers('Juan') ■■■ Selecciona ciudad: onChange('site',
'Medellín') ■■■ Dispara: getAdminSpacesByCity('Medellín') ■■■ Dispara:
getAdminEquipmentByCity('Medellín') ■■■ Selecciona espacio: onChange('space', 'Sala A1') ■■
■■ Selecciona fecha: onChange('date', '2026-04-15') ■■■ Ingresa horario:
onChange('startTime', '14:00') ■■■ Ingresa horario: onChange('endTime', '15:30') ■■■
User: Click "Crear Reserva" ■■ validate() → valida campos obligatorios ■■■ Si hay errores:
muestra mensajes en línea ■■■ createAdminReservation() → ■ POST /bookings/reservations HTTP
REQUEST: POST /bookings/reservations Authorization: Bearer <JWT_TOKEN> Content-Type:
application/json { "targetUserId": 123, "spaceId": 101, "startAt": "2026-04-15T14:00", "endAt":
"2026-04-15T15:30", "attendeesCount": 5, "equipmentIds": [1, 3] } BACKEND:
BookingController.createReservation() ■■ @AuthenticationPrincipal user (admin token) ■■
resolveEffectiveUserId(user, targetUserId=123) ■■■ Como user.hasRole("ADMIN"): retorna 123 ■■
Construye: CreateReservationCommand(userId=123, spaceId=101, ...) ■■ Invoca:
bookingUseCase.createReservation(command) ■■ BookingApplicationService.createReservation() ■■
VALIDACIÓN 1: Espacio existe ■■ VALIDACIÓN 2: Rango de fechas válido ■■ VALIDACIÓN 3: Sin
solapamiento ■■■ Query: SELECT * FROM reservations ■■ WHERE space_id=101 ■■ AND status IN
('pending', 'confirmed', 'in_progress') ■■ AND start < '2026-04-15T15:30' ■■ AND end >
'2026-04-15T14:00' ■■■ Resultado: 0 rows (sin solapamiento) ■■■ Validación pasa ■■ VALIDACIÓN
```

4: Equipos válidos ■ ■■ Equipos 1, 3 existen y pertenecen a Medellín ■ ■■ INSERT en BD: ■ ■■
INSERT INTO reservations (user_id, space_id, start_datetime, ...) ■ ■ VALUES (123, 101,
'2026-04-15 14:00', ..., 'pending') ■ ■■ Retorna: id=500 ■ ■■ Publica evento: ■ ■■
ReservationCreatedEvent → RabbitMQ ■ ■■ Retorna: Reservation (id=500, status='pending') HTTP
RESPONSE (201 Created): { "ok": true, "data": { "id": 500, "userId": 123, "spaceName": "Sala A1",
"startDatetime": "2026-04-15T14:00", "endDatetime": "2026-04-15T15:30", "status": "pending",
"attendeesCount": 5, ... } } FRONTEND: ■■ setSuccessMessage("Reserva creada exitosamente") ■■
Muestra toast por 3 segundos ■■ reset() → limpia formulario ■■ setIsCreateModalOpen(false) →
cierra modal ■■ loadAdminReservations(filters, 0) ■ ■■ Recarga listado ■ ■■ Nueva reserva
aparece en top (ordenamiento DESC por fecha) ■■ User ve su reserva creada en tiempo real PARALELO
(WebSocket - Notifications-Service): ■■ NotificationsService recibe: ReservationCreatedEvent ■■
Publica: /topic/events.bookings.reservation.created ■■ Frontend (si suscrito): ■ ■■ Recibe evento
en tiempo real ■ ■■ Podría actualizar UI sin necesidad de reload

Síntesis de Arquitectura Integrada

[illegible]

Fin del Análisis Técnico Reorganizado

Fecha: 2026-04-08

Enfoque: Segregación por Servicio, Principios SOLID Desglosados, Patrones con Archivos Específicos